Box Man: Static recompilation of Game Boy games with LLVM

Team Gloog 

12 August 2026

1 Introduction

Building software-based emulators has always been an interesting exercise for recreational and hobbyist programmers. There is something satisfying about running old, nostalgic games like The Legend of Zelda, Super Mario Bros., or Pokémon Red/Blue/Yellow through an emulator you built yourself.

Earlier this year, one of us got nerd-snipped into emulation development (commonly referred to as "emudev") by this site called Visual 6502, which is a browser-based transistor-level 6502 emulator. From then on, we have built a couple of small-scale emulators for old 2D-based retro game systems like CHIP-8 and NES.

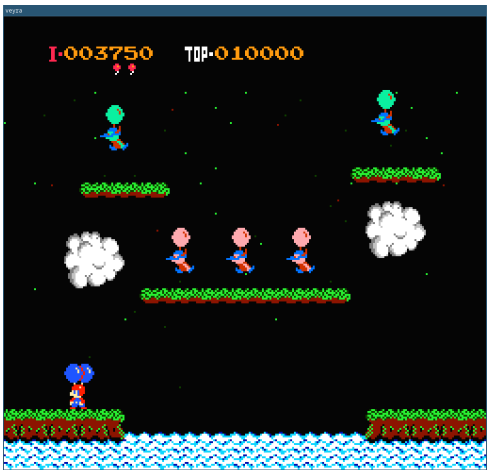

Figure 1: Balloon Fight running on tiny.nes

We wanted to build another emulator for the Game Boy, but this time we didn't want to build just another generic interpreter. Around then, we came across a blog post by Andrew Kelley titled "Statically Recompiling NES Games into Native Executables with LLVM and Go", and that's where the idea for Box Man came from.

Within the reverse engineering ecosystem, *manual* static recompilation of retro games is a very old and well-established practice. Some of the most popular community efforts in this context are Super Mario 64 decompilation and Pokémon Red/Blue disassembly. The researchers had to painstakingly go through the binaries of these games, analyze them and turn them into readable and byte-matching source code. These community efforts helped in improving the documentation of these systems which led to rise to automated static recompiler tools like N64Recomp which is being used by Zeld64Recomp project to statically recompile The Legend of Zelda: Majora's Mask game.

Box Man is a tool that takes in Game Boy ROMs and emits LLVM IR, which can then be compiled into a native executable using tools like `clang`. The resulting binary plays that specific game natively, without needing an external Game Boy emulator at runtime. It's basically equivalent to ahead-of-time compilation of Game Boy ROMs.

2 Architecture

Box Man isn't a fully self-contained static recompiler. It focuses mainly on statically recompiling the CPU i.e. the actual instruction stream the game executes. Apart from CPU, Game Boy contains other hardware components like PPU, APU, timer and joypad. All of them play an important role for actually making the game playable. In Game Boy, memory-mapped I/O is used i.e. CPU communicates with other hardware components by reading and writing to special memory addresses. But none of the aforementioned hardware components are *actually* part of the instruction stream. Due to this, we implement the behavior of the them using a small runtime library, which is linked at compile time alongside the recompiled CPU code.

Firstly, the ROM (or cartridge) file header is parsed to fetch the metadata such as the game's title, ROM and RAM size, and checksums for integrity. After that, control-flow analysis is done to recover the structure of the game's code. Starting from known entry points, which include the cartridge's boot address and the fixed interrupt vector address. Box Man recursively walks through the code, splitting into different basic blocks at every branch, call, and return. Anything which wasn't encountered by this recursive walk is treated as data rather than code. After the basic blocks are generated, each one of them is lowered into LLVM IR, where the entire game loop is represented as a single function. Instructions which try to interact with other hardware components using memory-mapped I/O are processed by the runtime library under the hood for tasks such as rendering the game state, taking the input from keyboard, playing sound, etc.

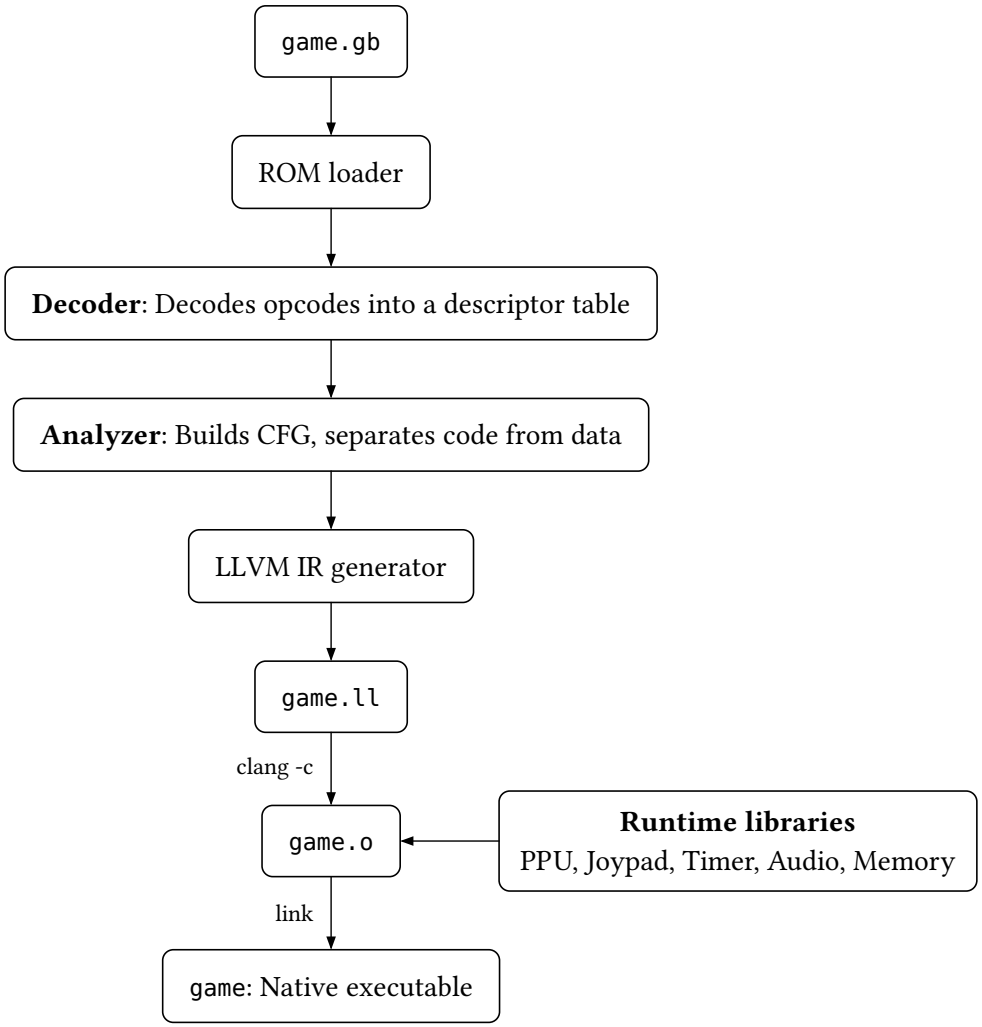

Figure 2: Box Man's static recompilation pipeline